การยางแห่งประเทศไทย
Rubber Authority of Thailand

Ditruz
IT Services Provider

Deploy .NET & Vue Application

ระยะเวลา: ~ 1 ชั่วโมง

เนื้อหาภายในคู่มือ

บทเรียน	จุดประสงค์
บทที่ 1: ภาพรวม Web Application	เข้าใจภาพรวม โครงสร้าง และองค์ประกอบก่อนลงมือ
บทที่ 2: Server และ Linux คืออะไร	รู้จัก Environment ที่จะทำงาน (บ้านของโปรแกรม)
บทที่ 3: SSH และการเข้า Server	เข้าควบคุม Server ได้ด้วยตัวเองอย่างปลอดภัย
บทที่ 4: .NET คืออะไร ทำงานอย่างไร	เข้าใจกระบวนการทำงานของสิ่งที่จะนำไป Deploy
บทที่ 5: Deploy คืออะไร ทำไมถึงสำคัญ	เข้าใจหัวใจของ Workshop และความสำคัญในสายงาน IT

ภาพรวม Web Application

เว็บไซต์ที่เราใช้งานอยู่ทุกวันมี 2 ประเภทหลัก

เว็บไซต์แบบ Static

- เนื้อหาไม่เปลี่ยนแปลง
- เปรียบเสมือนโบรชัวร์ออนไลน์
- ตัวอย่าง: หน้าแนะนำบริษัท, บล็อกอ่านข่าว
- ไม่ต้องมี logic การคำนวณซับซ้อน

Web Application (Dynamic)

- เนื้อหาเปลี่ยนตาม User และข้อมูล
- เปรียบเสมือนโปรแกรมที่รันบนอินเทอร์เน็ต
- ตัวอย่าง: ระบบ HR, ระบบจองห้อง, อีเมล
- **Workshop นี้เราจะ Deploy แบบนี้** (มี Logic & Database)

ส่วนประกอบของ Web App

เปรียบเทียบการทำงานกับ "ร้านอาหาร"

ลูกค้า (Browser)

คนที่มานั่งกินอาหาร ส่งคำสั่ง (Request) ผ่านหน้าจอ

พนักงาน (Web Server)

คนที่รับออเดอร์และนำอาหารไปส่งกลับให้ลูกค้า

ห้องครัว (.NET App)

ที่ปรุงอาหารจริง ๆ ประมวลผลตาม Logic ที่เขียนไว้

คลัง (Database)

ที่เก็บวัตถุดิบและข้อมูลทั้งหมดของระบบ

API คืออะไร?

API คือ "เมนูอาหาร" ของโปรแกรม
บอกว่าคุณสามารถสั่งอะไรได้บ้าง และแต่ละอย่างราคาเท่าไหร่

การสื่อสารระหว่างระบบ
API บอกว่า "คุณสามารถขอข้อมูลอะไรได้บ้าง และต้องส่งอะไรมาให้เราประมวลผล"

ตัวอย่าง GET Request:
GET /api/employees → ขอรายชื่อพนักงานทั้งหมด

ตัวอย่าง POST Request:
POST /api/leave → ยื่นคำขอลา (ส่งวันที่มาด้วย)

Server และ Linux คืออะไร?

Server คืออะไร

คอมพิวเตอร์ที่เปิดตลอด 24 ชั่วโมง รอรับ Request จาก User

เปรียบเสมือน **"อพาร์ตเมนต์"**

- **อาคาร (Physical Server):** เครื่องคอมพิวเตอร์จริง ๆ ที่ห้อง Data Center
- **ห้องเช่า (Virtual Machine / VM):** คอมพิวเตอร์จำลองในเครื่องนั้น
- **ผู้เช่า (Application):** โปรแกรมของเราที่รันอยู่ใน VM

ทำไมต้อง VM?

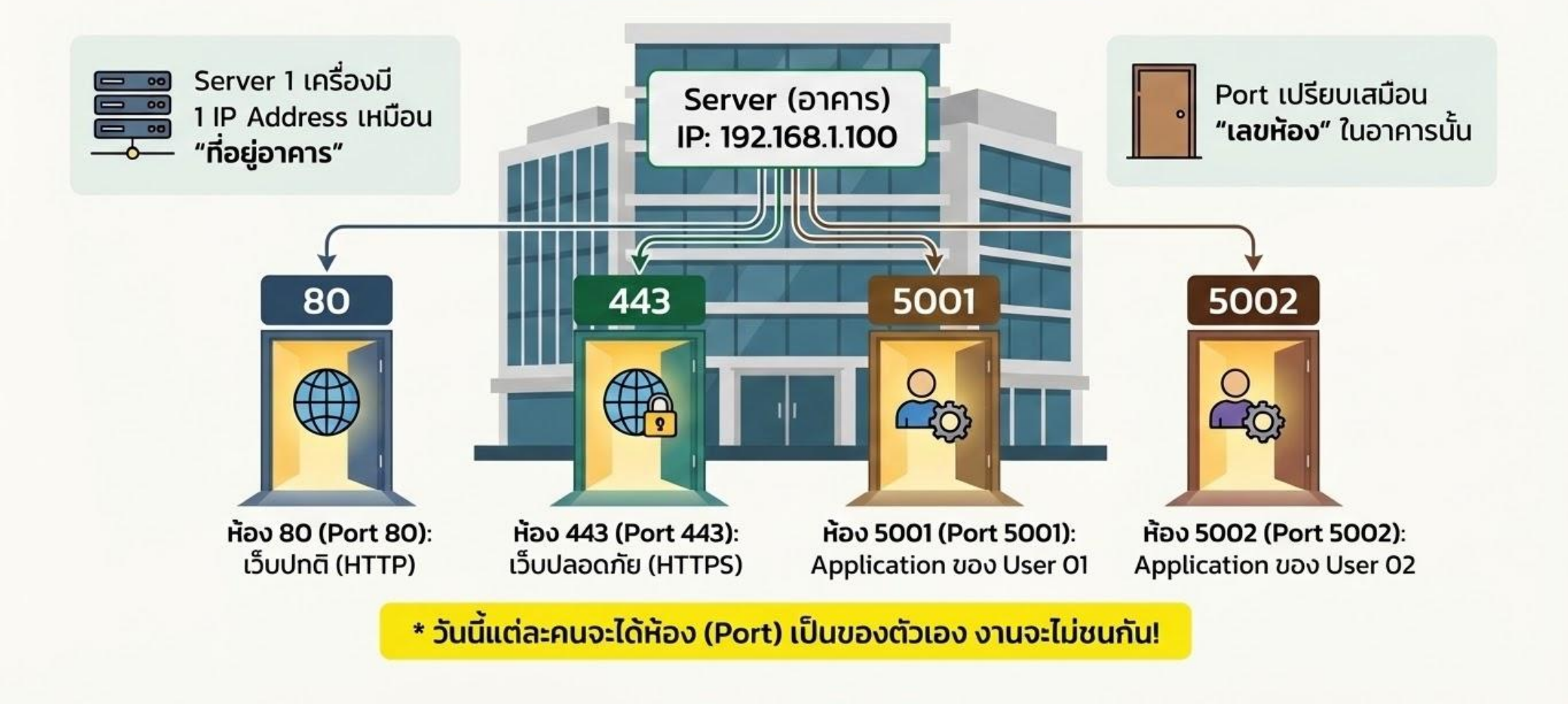
- **ประหยัด:** รัน Server หลายตัวบน Hardware เดียว
- **ยืดหยุ่น:** สร้าง, ลบ, Copy VM ได้ง่าย
- **ปลอดภัย:** แต่ละ VM แยกการทำงานกันอิสระ
- **Cloud พื้นฐาน:** ใช้จริงบน AWS (EC2), Azure

ทำไมถึงใช้ Linux (Ubuntu)?

Windows	Linux (Ubuntu)
มีหน้าต่าง (GUI), ใช้เมาส์คลิกได้ง่าย	**ส่วนใหญ่ใช้ Command Line (พิมพ์คำสั่ง)**
มีค่าใช้จ่าย (ลิขสิทธิ์แพง)	**ฟรี / Open Source**
นิยมใช้บนเครื่อง PC ทั่วไป	**นิยมใช้บน Server มากที่สุดในโลก**
อัปเดตระบบมักต้อง Restart บ่อย	**รันได้ยาวนานโดยไม่ต้อง Restart**
ใช้ทรัพยากรเครื่อง (RAM/CPU) ค่อนข้างมาก	**เบา ประหยัดทรัพยากร (Lightweight)**

* FACT เว็บไซต์ส่วนใหญ่ในโลก (กว่า 70%) รันบน Linux Server

Port คืออะไร?


Server 1 เครื่องมี
1 IP Address เหมือน
"ที่อยู่อาคาร"
Server (อาคาร)
IP: 192.168.1.100
Port เปรียบเสมือน
"เลขห้อง" ในอาคารนั้น
80
443
5001
5002
ห้อง 80 (Port 80):
เว็บปกติ (HTTP)
ห้อง 443 (Port 443):
เว็บปลอดภัย (HTTPS)
ห้อง 5001 (Port 5001):
Application ของ User 01
ห้อง 5002 (Port 5002):
Application ของ User 02
* วันนี้แต่ละคนจะได้ห้อง (Port) เป็นของตัวเอง งานจะไม่ชนกัน!

SSH: กุญแจสู่โลก Linux

Secure Shell (SSH)

คือวิธีที่เราใช้ "เข้าไปควบคุม" Server ระยะไกลผ่านหน้าจอ Command Line ข้อมูลถูกเข้ารหัส (Encrypt)

อุปมาอุปไมย: โทรศัพท์ + รหัส PIN

1. เรา **โทรหา** Server บอก IP Address (เบอร์โทร)
2. Server **ถามชื่อ** (Username)
3. เรา **ใส่รหัสผ่าน** (Password)
4. เราเข้าไปอยู่ใน Server ได้ ราวกับนั่งอยู่หน้าเครื่อง

```
# วิธีเชื่อมต่อ SSH
user@local:~$ ssh user01@79.108.225.69
user01@79.108.225.69's password: (พิมพ์รหัสจะไม่แสดงบนจอ)

Welcome to Ubuntu 22.04 LTS (GNU/Linux)
Last login: Mon Oct 24 10:00:00 2023
user01@workshop-deploy-1:~$ _
```

คำสั่ง Linux ที่จำเป็น (สำหรับ Workshop)

คำสั่ง	ความหมาย (เปรียบเทียบ)
pwd	บอกว่าตอนนี้อยู่ที่ Folder ไหน (ถามว่า "ฉันอยู่ที่ไหน?")
ls	แสดงรายการไฟล์ใน Folder ปัจจุบัน (เหมือน dir ใน Windows)
cd [ชื่อโฟลเดอร์]	เปลี่ยนเข้า Folder (cd = change directory) / cd .. กลับไปหน้า
git clone [url]	Download โค้ดจาก Git มาไว้ที่ Server
deploy	**คำสั่งพิเศษ:** Build + Start App อัตโนมัติ
tail -f /tmp/deploy.log	ดู Log แบบ Real-time เพื่อตรวจสอบสถานะ/ปัญหา

.NET คืออะไร ทำงานอย่างไร

.NET คือ Platform ของ Microsoft ที่รันได้หลาย OS เปรียบเสมือน "เครื่องยนต์"

Code (Blueprint)

โค้ด .NET ที่ Developer เขียน (ไฟล์ .cs, .csproj) เปรียบเหมือน "พิมพ์เขียวของรถ" ยังวิ่งไม่ได้ต้องสร้างก่อน

.NET SDK (Tools)

ชุดเครื่องมือสำหรับ "สร้างรถ" ใช้คำสั่ง Build เพื่อแปลงพิมพ์เขียว (Code) ให้กลายเป็น Binary (รถยนต์จริง)

.NET Runtime

เครื่องยนต์จริง ๆ ที่ติดตั้งไว้บน Server เพื่ออ่าน Binary และขับเคลื่อนโปรแกรมให้ทำงานรับ Request ได้

Deploy คืออะไร ทำไมถึงสำคัญ?

Deploy = การ "**เปิดร้าน**" นำโปรแกรมไปวางและเปิดให้ใช้งานบน Server จริง

แบบ Manual (ทำมือ)

- Developer push code ขึ้น Git
- **ต้อง SSH เข้า Server เอง**
- รันคำสั่ง Build + Deploy เอง
- เหมาะกับ: การเรียนรู้, Server ขนาดเล็ก
- *** Workshop นี้ใช้วิธีนี้**

แบบ CI/CD (อัตโนมัติ)

- Developer push code ขึ้น Git
- **ระบบทำงานอัตโนมัติทันที**
- Build, Test, Deploy อัตโนมัติ
- เหมาะกับ: Production จริง, ทีมขนาดใหญ่
- เป็นทักษะต่อยอดในสายงาน DevOps

Workshop: ลงมือปฏิบัติ (Step 1-2)

Step 1: SSH เข้า Server

เข้าสู่ Server ด้วย Username และรหัสผ่านที่ได้รับ

```
> ssh userXX@79.108.225.69
Password: [พิมพ์รหัสผ่าน]
Welcome to .NET + Vue Workshop! 🚀
Port : 50XX
```

Step 2: Clone Project

ดาวน์โหลดโค้ดตั้งต้นจาก Git มายัง Server

```
> git clone [GIT_URL_ของ_PROJECT]
> ls
# ตรวจสอบว่ามีโฟลเดอร์โปรเจกต์ปรากฏขึ้น
```

Workshop: ลงมือปฏิบัติ (Step 3-4)

Step 3: เข้า Folder Project

ย้ายตัวเองเข้าไปในโฟลเดอร์ที่มี Source Code

```
> cd [ชื่อ_FOLDER_PROJECT]
> ls
# ควรเห็นไฟล์ .csproj
```

Step 4: Deploy!

ใช้คำสั่งพิเศษเพื่อ Build และ Start Application

```
> deploy
[1/3] Building ...
[2/3] Stopping old process ...
[3/3] Starting application ...
✅ Deploy สำเร็จ! URL: http://[VM_IP]:[PORT]
```

สรุป Workshop: 4 คำสั่งที่ต้องจำ

1

`ssh user@ip`

เข้าควบคุม Server ด้วยรหัสผ่านที่ได้รับ

2

`git clone`

ดาวน์โหลด Source Code ลงมาที่ Server

3

`cd folder`

ย้ายที่ทำงานเข้าไปในโฟลเดอร์โปรเจกต์

4

`deploy`

Build และ Start อัตโนมัติ ได้ URL พร้อมใช้งาน

อัปเดตโค้ดใหม่: พิมพ์ git pull && deploy เพื่ออัปเดตระบบทันที

Deploy Vue Frontend

อีกครึ่งหนึ่งของ Web Application ที่ขาดไม่ได้

Frontend คืออะไร? ต่างกับ Backend อย่างไร?

Backend (.NET API) = ห้องครัว

- ทำงานอยู่เบื้องหลัง ไม่เปิดเผยให้เห็นโดยตรง
- เตรียมวัตถุดิบ ปรุงอาหารตามคำสั่ง (ประมวลผล Logic)
- เน้นความถูกต้อง ปลอดภัย และมีประสิทธิภาพ

Frontend = หน้าร้าน

- ส่วนที่ **ลูกค้าเห็นและสัมผัสโดยตรง** (UI/UX)
- แสดงข้อมูล, รับ Input จาก User, และเรียก API
- **"ครัวเก่งแค่ไหน ถ้าหน้าร้านไม่มี ลูกค้าก็เข้าไม่ได้"**

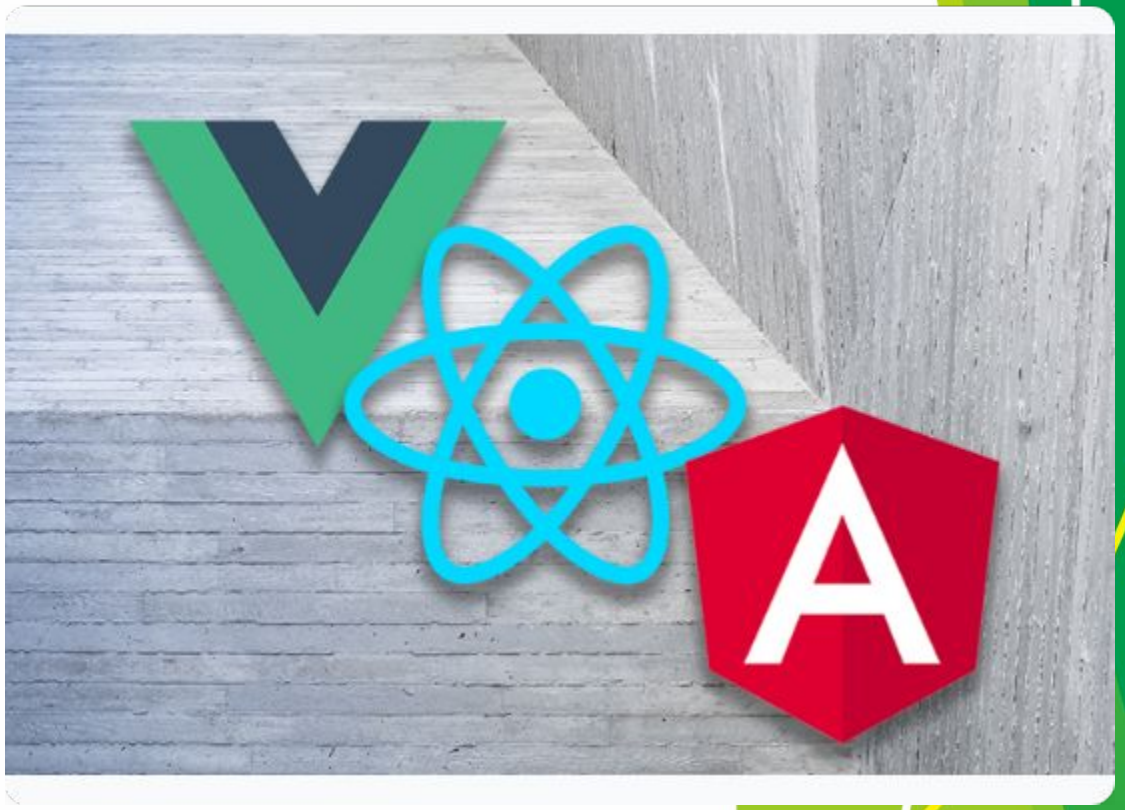
* **React, Vue, Angular** คือ Framework ยอดนิยมสำหรับสร้าง Frontend ทั้งสามตัวทำงานเหมือนกัน ต่างกันแค่ Syntax และ Style การเขียน

กระบวนการ Build ของ Frontend

ทำไมต้อง Build?

Code ที่ Developer เขียน (JSX, TypeScript, Vue SFC) Browser อ่านตรง ๆ ไม่ได้ ต้องนำมา "Build" เพื่อแปลงเป็น HTML, CSS, JavaScript พื้นฐานที่ Browser เข้าใจ

1. **npm install**
   ดาวน์โหลด Package/Library ที่ Project ต้องใช้
2. **npm run build**
   แปลงโค้ดจาก React/Vue/Angular ให้เป็น Static Files
3. **ได้ Folder dist/ หรือ build/**
   นี่คือไฟล์ที่พร้อม Serve (พร้อมเสิร์ฟ)
4. **ใช้ Web Server เสิร์ฟไฟล์**
   ใน Workshop นี้เราใช้คำสั่ง serve เพื่อเป็น Web Server



ความสัมพันธ์ระหว่าง Frontend กับ Backend

Frontend (React/Vue) = รีโมท
ส่วนที่ผู้ใช้จับและกดปุ่มสั่งการ

Backend (.NET API) = ทีวี
ส่วนที่รับคำสั่ง ประมวลผล และเตรียมสัญญาณภาพแสดงผล

กระบวนการทำงานจริง:

[ผู้ใช้] กดปุ่ม 'โหลดข้อมูล' บนหน้าเว็บ

↓

[Frontend] ส่ง Request เรียกไปที่ API

↓

[.NET API] ดึงข้อมูลจาก DB แล้วส่งผลกลับมาแสดงบนหน้าจอ

ไฟล์สำคัญ & การตั้งค่า API URL

ไฟล์ / Folder	ความหมาย
package.json	สารบัญโปรเจกต์ - บอกว่าใช้ React/Vue/Angular และเวอร์ชันเท่าไร
node_modules/	Folder ที่เก็บ Library ทั้งหมด (ไฟล์เยอะและใหญ่มาก *ห้ามอัปขึ้น Git*)
src/	Folder ที่ Developer เขียน Code ของแอปพลิเคชัน
dist/ หรือ **build/**	Output หลังทำการ Build เป็นไฟล์ที่จะ Serve ให้ Browser
.env หรือ **.env.production**	ไฟล์สำหรับตั้งค่า Environment เช่น **API URL** ที่ Frontend จะต้องเรียก

Workshop: ลงมือ Deploy Frontend

ขั้นตอนการปฏิบัติ:

- **Step 1:** SSH เข้า Server (ถ้ายังไม่ได้เข้า)
- **Step 2:** Clone Frontend Project (จาก Git)
- **Step 3:** cd เข้า Folder ที่มี package.json
- **Step 4:** พิมพ์ deploy แล้วรอระบบทำงานอัตโนมัติ

```
# โคลนและเข้าไปในโฟลเดอร์โปรเจกต์
> git clone [GIT_URL_FRONTEND]
> cd [frontend-folder]

# สั่ง Deploy
> deploy
[Auto-Detect] Found package.json
Deploying React/Vue/Angular App ...
Running npm install ... (อาจใช้เวลา 1-2 นาที)
Running npm run build ...
Starting static server on port 6001 ...
✅ Frontend Deploy สำเร็จ! URL: http://[VM_IP]:6001
```

การตรวจสอบสถานะ & Error ที่พบบ่อย

Error / อาการที่พบ	สาเหตุ / วิธีแก้ไข
npm ERR! / Build failed	Code มี Error หรือตั้งค่า Dependency ผิด → **แจ้ง Developer**
หน้าขาว / Blank Page	Build Path ผิด → ดู Log: tail -f /tmp/fe_userXX.log
ข้อมูลไม่ขึ้น แต่ UI โหลดได้	ตรวจสอบ .env.production ของ Frontend → **API URL อาจผิด**
CORS error (ใน Browser Console)	Backend ไม่อนุญาตให้ Frontend โดเมนนี้เรียกข้อมูล → แจ้ง Developer
ERR_CONNECTION_REFUSED	Frontend ไม่ได้รัน หรือพิมพ์ Port ผิด → ใช้คำสั่ง mystatus ตรวจสอบ

```
> mystatus
[OK] .NET API : RUNNING → http://[VM_IP]:5001
[OK] Frontend : RUNNING → http://[VM_IP]:6001
```

Full Stack Deploy - สรุปคำสั่งทั้งหมด

1. Deploy .NET API

```
cd && deploy
```

→ URL: http://VM_IP:5001

2. Deploy Frontend

```
cd && deploy
```

→ URL: http://VM_IP:6001

3. อัปเดต Code ใหม่

```
git pull && deploy
```

(ใช้ได้ทั้ง API และ Frontend โฟลเดอร์)

เครื่องมือ Troubleshoot อื่น ๆ

mystatus stopapp tail -f /tmp/api_userXX.log tail -f /tmp/fe_userXX.log

Q&A

Questions & Answers - Thank you for participating!